Presenter 2 — Harsh: guardian and defence (5:00–10:00)

Your question: Can we detect or disrupt the hidden timing *without* accusing ordinary traffic or delaying it too much? You own **Guardian**, **Detectors**, and **Stress matrix**. You do not need to re-teach the sender's 0/1 encoding; the first speaker has done that.

Start screen: Guided case — Guardian. **End screen:** Guided case — Locked note. Leave the mouse with Aarush.

Your five-minute route

5:00–5:45 — Explain what a guardian actually judges

Point to the three rule choices. Say:

> "We should not declare an attack from one unusual pause. This guardian judges a small group of 32 requests. It measures how much their gaps vary, then compares that with a limit. We test three ways to choose the limit: one fixed number, a limit learned from ordinary traffic under the same conditions, and a limit learned only on a clean network."

Plain terms: a *window* is one group of requests; a *baseline* is a normal reference used to choose a limit; *standard deviation* means how much the pauses wobble around their average.

5:45–6:45 — Compare the rules, including mistakes

Begin on `Same conditions`. Point to **Messages caught** and **False alarms** together. On the default example they show **10/10 caught** and **3/10 ordinary windows falsely flagged**. Click **Fixed limit**: it usually shows **0/10 caught, 0/10 false alarms** on this default example. Click **Clean-only baseline**: it shows **10/10 caught but 10/10 false alarms**. Return to **Same conditions**.

> "If a rule never sounds the alarm, its false-alarm rate looks perfect but it misses the message. If it alarms on everything, it catches the message but becomes useless for ordinary users. The same-conditions baseline is more sensible here, yet even it still wrongly flags some normal traffic. These figures describe this small simulated set, not an accuracy guarantee for the Internet."

Read the live figures if the condition or dataset was changed. Do not memorize a percentage without its numerator and denominator.

6:45–7:20 — Make the statistics visible

Point to `Verdict by observation window`. Red squares in the ordinary row are false alarms; red squares in the message row are detections.

> "These squares are the actual decisions behind those percentages. The ordinary row shows the cost of mistakes; the message row shows what was caught. That is why we report both numbers together."

7:20–8:05 — Show the second defence and its cost

Point to `Try a second defence: pace the packets`. At the default **500 ms** waiting room, the screen shows about **90% bits read before**, **57% after**, and **+470 ms** added wait to ordinary pings. Move the slider briefly to **0 ms**: no added wait and no disruption. Return to **500 ms**.

> "This model releases requests on steadier one-second slots when it can. It cannot send a packet before it arrives; it only holds it. More waiting can damage the hidden timing, but it also delays innocent pings. The relay is a browser model, not a deployed router."

8:05–9:10 — Show the full detector workbench

Click `Detectors` in the top navigation. Point to the short explanation at the top of **Settings**, then the three main choices: **Network variation**, **Message subtlety**, and **Learn normal from** (Same network / Clean network). Point to the four result tiles, then scroll once to **Verdict per window**. No need to tune anything live.

> "This is the full instrument panel behind the guided guardian. By default it asks three questions: how noisy is the network, how subtle is the message, and did we learn normal traffic under the same network or a clean one? A good detector catches the hidden-message windows and leaves ordinary traffic alone, so we read detection and false alarms together. Everything else, data source, delay model, window size, percentile, fixed thresholds and features, is under Advanced options. The guided scene uses a narrow, readable subset of this same analysis."

Optional, only if you have 15 seconds: click **Advanced options** to show the deeper controls exist, or **Show** on **ROC curve** (it starts collapsed, like **Which feature separates best**).

If someone asks about the ROC curve: it plots detection against false alarms as a threshold changes. It does **not** turn this small simulated dataset into real-world validation.

9:10–9:45 — Show the broader test grid

Click `Stress matrix`. Only **Metric** and **Detector** are shown; window, delay model and data source sit under **Advanced filters**. Point to the **How to read the grid** box, then across the message timings (rows) and network conditions (columns). Darker means a higher value. Optionally change **Metric** from `detection rate` to `false-positive rate` if you have at least 20 seconds; the box then says dark is bad. On a phone-sized screen the grid turns sideways.

> "One attractive result can hide a fragile method. This grid asks what happens when we make the timing subtler or add delay and loss. We can switch the metric to see detection, false alarms, or decoding accuracy. This is where we look for failure cases, not just the best cell."

9:45–10:00 — Hand off cleanly

Click `Guided case`, then chapter `04 Locked note`. Say:

> "The guardian can see and disrupt timing, but it has not read the note itself. Aarush will show what encryption changes, what it costs, and what our evidence supports."

Stop speaking and hand him the laptop.

Your three memory anchors

1. **Detection rate and false-alarm rate must be read together.**
2. **A normal baseline should match the conditions it will face.**
3. **A pacing defence has a legitimate-traffic delay cost.**

If you get a question

- **“What exactly is the learned limit?”** The guided baseline uses the 95th-percentile standard deviation from separate ordinary 32-request windows; it alarms when a test window exceeds it.
- **“Why 32?”** It is the guided case's observation size, a trade-off between reacting quickly and collecting enough gaps. The full Detectors tool lets us change it; we do not claim 32 is universally optimal.
- **“Is 100% detection a proven result?”** No: here it means 10 of 10 message windows in one controlled simulated condition. Also state the false alarms and the absence of real captures.
- **“Did you install this relay on a router?”** No. The algorithm is a causal model in the browser. A real deployment would need latency and throughput measurements on lab hardware.

If the clock is running out: show one comparison in Guardian, the pacing cost, then briefly expose the Detectors and Stress matrix pages (their default screens are enough; skip Advanced options). Keep the 10:00 handoff; Aarush needs his full five minutes.